

AFRILANG Stdlib

std/c/graphbfs

Français. Bibliothèque AFRILANG 'std/c/graphbfs' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bfsDistances, bfsOrder, bfsReachable, bfsLayers, isConnected, shortestPathLen, multiSourceBfs.

```
'import "std/c/graphbfs"' · 'use graphbfs'
```

- 'bfsDistances(adj list number, n number, start number) → list number'
- 'bfsOrder(adj list number, n number, start number) → list number' -
- 'bfsReachable(adj list number, n number, start number) → number' -
- 'bfsLayers(adj list number, n number, start number) → list number' -
- 'isConnected(adj list number, n number) → bool' - 'shortestPathLen(adj list nu

English. AFRILANG library 'std/c/graphbfs' (tier complex, category complex). Exports 7 function(s): bfsDistances, bfsOrder, bfsReachable, bfsLayers, isConnected, shortestPathLen, multiSourceBfs.

```
'import "std/c/graphbfs"' · 'use graphbfs'
```

- 'bfsDistances(adj list number, n number, start number) → list number'
- 'bfsOrder(adj list number, n number, start number) → list number' -
- 'bfsReachable(adj list number, n number, start number) → number' -
- 'bfsLayers(adj list number, n number, start number) → list number' -
- 'isConnected(adj list number, n number) → bool' - 'shortestPathLen(adj list number, n numbe

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/graphbfs' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bfsDistances, bfsOrder, bfsReachable, bfsLayers, isConnected, shortestPathLen, multiSourceBfs.

'import "std/c/graphbfs"' · 'use graphbfs'

```
- `bfsDistances(adj list number, n number, start number) → list number`
- `bfsOrder(adj list number, n number, start number) → list number`
- `bfsReachable(adj list number, n number, start number) → number`
- `bfsLayers(adj list number, n number, start number) → list number`
- `isConnected(adj list number, n number) → bool`
- `shortestPathLen(adj list number, n number, start number, goal number) → number`
- `multiSourceBfs(adj list number, n number, sources list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphbfs"
use graphbfs
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- bfsDistances(adj list number, n number, start number) → list•
bfsOrder(adj list number, n number, start number) → list•
bfsReachable(adj list number, n number, start number) → number•
bfsLayers(adj list number, n number, start number) → list•
isConnected(adj list number, n number) → bool•
shortestPathLen(adj list number, n number, start number, goal number) → number•
multiSourceBfs(adj list number, n number, sources list number) → list

4. Exemples d'utilisation

```
import "std/c/graphbfs"
use graphbfs
```

```
create result = bfsDistances(/* args */)
say result
```

```
create result = bfsOrder(/* args */)
say result
```

```
create result = bfsReachable(/* args */)
say result
```

```
create result = bfsLayers(/* args */)
say result
```

```
create result = isConnected(/* args */)
say result
```

```
create result = shortestPathLen(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/graphbfs"`` en tête de fichier.
- Lancez ``afirang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module graphbfs

```
function bfsDistances(adj list number, n number, start number) returns list number
end
```

```
function bfsOrder(adj list number, n number, start number) returns list number
end
```

```
function bfsReachable(adj list number, n number, start number) returns number
end
```

```
function bfsLayers(adj list number, n number, start number) returns list number
end
```

```
function isConnected(adj list number, n number) returns bool
end
```

```
function shortestPathLen(adj list number, n number, start number, goal number)
  returns number
end
```

```
function multiSourceBfs(adj list number, n number, sources list number) returns list
  number
end
end
```

English

1. Overview

AFRILANG library 'std/c/graphbfs' (tier complex, category complex). Exports 7 function(s): bfsDistances, bfsOrder, bfsReachable, bfsLayers, isConnected, shortestPathLen, multiSourceBfs.

```
'import "std/c/graphbfs"' · 'use graphbfs'
```

```
- `bfsDistances(adj list number, n number, start number) → list number`
- `bfsOrder(adj list number, n number, start number) → list number`
- `bfsReachable(adj list number, n number, start number) → number`
- `bfsLayers(adj list number, n number, start number) → list number`
- `isConnected(adj list number, n number) → bool`
- `shortestPathLen(adj list number, n number, start number, goal number) → number`
- `multiSourceBfs(adj list number, n number, sources list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphbfs"
use graphbfs
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- bfsDistances(adj list number, n number, start number) → list•
bfsOrder(adj list number, n number, start number) → list•
bfsReachable(adj list number, n number, start number) → number•
bfsLayers(adj list number, n number, start number) → list•
isConnected(adj list number, n number) → bool•
shortestPathLen(adj list number, n number, start number, goal number) → number•
multiSourceBfs(adj list number, n number, sources list number) → list

4. Usage examples

```
import "std/c/graphbfs"
use graphbfs
```

```
create result = bfsDistances(/* args */)
say result
```

```
create result = bfsOrder(/* args */)
say result
```

```
create result = bfsReachable(/* args */)
say result
```

```
create result = bfsLayers(/* args */)
say result
```

```
create result = isConnected(/* args */)
say result
```

```
create result = shortestPathLen(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/graphbfs"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module graphbfs

```
function bfsDistances(adj list number, n number, start number) returns list number
end
```

```
function bfsOrder(adj list number, n number, start number) returns list number
end
```

```
function bfsReachable(adj list number, n number, start number) returns number
end
```

```
function bfsLayers(adj list number, n number, start number) returns list number
end
```

```
function isConnected(adj list number, n number) returns bool
end
```

```
function shortestPathLen(adj list number, n number, start number, goal number)
  returns number
end
```

```
function multiSourceBfs(adj list number, n number, sources list number) returns list
  number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/graphbfs"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/graphbfs/>

Source (excerpt)

```
module graphbfs

function bfsDistances(adj list number, n number, start number) returns list number
end

function bfsOrder(adj list number, n number, start number) returns list number
end

function bfsReachable(adj list number, n number, start number) returns number
end

function bfsLayers(adj list number, n number, start number) returns list number
end

function isConnected(adj list number, n number) returns bool
end

function shortestPathLen(adj list number, n number, start number, goal number)
  returns number
end

function multiSourceBfs(adj list number, n number, sources list number) returns list
  number
end
end
```